Decipher

A Foundational Architecture for Human Experience Computation

Authored by Raheem Asghar, Founder — Decipher Prepared for technical and strategic leadership

---

A Note From the Author

This document represents two years of foundational work pursuing a single question: why do organisations keep re-discovering the same problems with human experience, year after year, without ever structurally resolving them?

What began as a frustration with how CX systems were built evolved into an architectural thesis — and then into a working computational model. What follows is my attempt to explain what I have built, why I built it, and why I believe the timing makes it necessary now.

— Raheem Asghar

The Problem That Has No Name

For over two decades, organisations have known that certain operational surfaces determine whether a human experience succeeds or fails. They called them drivers. Drivers of satisfaction. Drivers of loyalty. Drivers of churn.

The intuition was always correct.

But drivers were never made real inside systems. They existed as analytical conclusions — thematic labels that drifted across quarters, reset with every reporting cycle, and vanished the moment the deck was filed. The same surfaces were re-identified, re-debated, and re-reported. Year after year.

This is not a failure of effort or intelligence. It is a structural failure.

Current experience systems were built to observe. They were never built to persist, govern, or learn. The result is that organisations have become extraordinarily

capable at describing their problems — and structurally incapable of accumulating knowledge about them.

Decipher addresses this at the architectural level.

The Core Claim

Experience has been observed. It has never been inhabited by machines.

A complaint about triage delays is not the phenomenon. The phenomenon is the condition of *Emergency Services → Triage Efficiency* as a real operational surface in the world. The complaint is evidence that updates the system's knowledge of that driver's state. But the driver exists independently, persists across time, and can improve or deteriorate regardless of whether anyone is currently reporting on it.

Current systems treat the signal as the unit of reality.

Decipher treats the driver as the unit of reality, and the signal as evidence that updates the driver's state.

That is a different ontology. Everything that follows depends on it.

The Primitive

Every durable computational system begins with a carefully chosen primitive — the smallest meaningful unit upon which all higher-order logic is built.

Decipher's primitive is the **Experience Driver**, expressed as *Category → Subcategory*:

- Emergency Services → Triage Efficiency
- Billing → Invoice Clarity
- Prescription Management → Refill Authorization
- Test Results → Communication Timeliness

An Experience Driver is not a theme, a score, a sentiment label, or a dashboard category. It is the underlying operational surface where experience occurs — the causal determinant that teams own, act on, and are accountable for.

The Experience Driver was always the right intuition in CX. What was missing was the computational substrate to make it real. Decipher provides that substrate.

The Three Structural Laws

To make Experience Drivers computationally real, three invariants are enforced at the schema level. These are not guidelines. They are the laws that transform a collection of records into an institutional ledger.

Identity Law. Each Experience Driver has exactly one canonical state container per analysis window. No duplicates. No parallel interpretations. No competing truths. One driver, one state.

Lineage Law. Every intervention is a timestamped state transition linked to the driver state that triggered it. Every action traces back to its cause. Every outcome traces forward to its consequence. Nothing happens without being recorded.

Memory Law. Every state transition retains referential integrity across time. No driver state is ever orphaned. Historical outcomes become priors for future decisions. The system does not reset.

These laws do for experience drivers what double-entry bookkeeping did for financial records: they impose a structural discipline that makes the domain governable rather than merely observable.

The Architecture

Decipher operates as a layered execution substrate. Each layer has a single responsibility and produces a single class of output.

Layer 1 — Signal Refinement. Raw human signals are processed through a governed taxonomy. One raw comment typically produces multiple structured Experience Driver mentions, each with its own affect axis, intent classification, journey stage, interaction moment, and behavioural context. Context is preserved, not collapsed. A single narrative about a hospital visit may contain four distinct operational realities, each mapped to a different driver, each requiring a different response.

Layer 2 — State Computation. All mentions for a given Experience Driver within a defined window are aggregated into a single Structured Experience Unit (SEU). The SEU is the authoritative state container for that driver. It computes:

- ERI (Emotional Resonance Index) — weighted polarity
- EVI (Emotional Volatility Index) — emotional dispersion
- RF (Recency × Frequency) — urgency composite
- Emotional State Band (ES1 Secure Loyalty through ES5 Critical Failure)
- Temporal intelligence — trajectory, not just snapshot

The SEU computation is governed and reproducible within a fixed mention set: given the same mentions, it always produces the same state. This is not inference. It is arithmetic over a structured input.

Layer 2.5 — Mission Generation. Orchestration Units (OUs) are generated by clustering mentions, not the aggregated SEU. This separation matters enough to warrant its own section, which follows.

Layer 3 — Execution. Each OU is converted into a locked Problem Statement that defines what needs to be addressed, why, and what the consequence of inaction is. After this point, reinterpretation is closed.

The Problem Statement initiates a PDCA cycle. In production, this cycle is not powered by generic intelligence. It is powered by the enterprise's own knowledge bases — clinical protocols, SOPs, regulatory requirements, ownership structures, budget constraints, and past intervention outcomes — accessed by an agent operating under organisational governance. The output is not an institutionally plausible recommendation. It is an institutionally true one.

Measurement. After each intervention, the SEU is recomputed. Before and after states are compared. Elasticity — the proportional state shift relative to effort — is calculated and stored. Impact is derived computationally, not self-reported.

Institutional Memory Engine (IME). All state transitions, interventions, and outcome evaluations are persisted with full referential integrity. The IME accumulates which interventions worked, under which SEU conditions, for which driver types, at which urgency levels. Past successes constrain future decisions. Patterns that humans forget, the system remembers.

The Decision That Makes the Architecture Work

The most important decision in this architecture is easy to miss in passing: **Orchestration Units are born from clustered mentions, not from the aggregated SEU.**

This is how the system reconciles two things that would otherwise contradict each other.

The driver identity must be stable. *Triage Efficiency* cannot mean one thing in January and something different in June, or longitudinal comparison becomes impossible. But the operational reality beneath that driver does change. In January the failures may cluster around elderly fall delays. In June they may cluster around trauma protocol activation. The driver label has not changed. The world beneath it has.

If OUs were generated from the aggregated SEU state, that shift would be invisible. The SEU would report "Triage Efficiency is in crisis," and execution would proceed against an abstract average that corresponds to no real operational pattern.

Generating OUs from clustered mentions resolves this. A single driver may simultaneously produce multiple OUs — each one a distinct, evidence-backed, behaviourally specific operational reality occurring under that driver at that time. The driver stays fixed. The SEU reflects aggregate state. The OUs reflect the actual shape of what is happening.

One driver → one SEU → many OUs. State and mission are deliberately separated. This separation is what allows the system to hold identity stable while the world beneath it moves.

The Emotional Relational Model

The ERM is the relational enforcement layer that makes the three laws hold at scale. It is not application logic. It is the schema-level substrate that enforces identity through uniqueness constraints, lineage through foreign keys, and memory through append-only event tables.

Because the ERM is a standard relational schema, its properties are queryable rather than inferred:

```
-- What was the state of Triage Efficiency on March 15?
SELECT eri, evi, rf, emotional_state_band
FROM seus
WHERE experience_driver_id = 'emergency_services_triage_efficiency'
  AND window_end_date = '2025-03-15';

-- Which interventions have historically worked for crisis-state drivers?
SELECT ou.opportunity_stream,
       AVG(eval.eri_delta) AS avg_improvement,
       COUNT(*) AS activation_count
FROM orchestration_units ou
JOIN ou_activations act ON act.orchestration_unit_id = ou.id
JOIN outcome_evaluations eval ON eval.ou_activation_id = act.id
JOIN seus s ON s.id = ou.seu_id
WHERE s.emotional_state_band = 'ES4_Active_Crisis'
  AND eval.outcome_classification = 'Successful'
GROUP BY ou.opportunity_stream
ORDER BY avg_improvement DESC;
```

These are not recommendations generated by a model. They are facts returned by a ledger. The distinction is the entire point of the architecture.

The properties that emerge directly from this structure:

- Full auditability — every action traces to its trigger and outcome
- Agent-native state — agents query shared state rather than re-inferring it
- Elasticity verification — structurally traceable evidence linking intervention to state change
- Model independence — the underlying LLM can be swapped without rewriting state
- Horizontal scalability — standard relational operations at any volume
- Zero lock-in — portable SQL schema; the organisation owns its ledger

---

Why This Is Buildable Now

Two different arguments have to hold for this architecture to exist today. They are worth separating.

The technical argument. Layer 1 requires reliable extraction of fully dimensional Experience Driver mentions from unstructured feedback at scale. That extraction requires semantic understanding that only became reliable with large language models. Everything above Layer 1 — the SEU, the OUs, the PDCA cycle, the IME — is standard relational computation that has been tractable for decades. The LLM is used precisely where language understanding is required, and its output is locked into deterministic relational structure before anything consequential happens. LLMs enable the architecture. They are not the architecture.

The demand argument. Before autonomous systems, humans mediated all experience action. Vague recommendations were acceptable because a human would add context and judgment. Agents cannot. They need stable canonical entities, shared truth they can query without re-inference, atomic executable missions, and persistent memory. The agentic wave has created a category of consumer that the current CX stack cannot safely serve. Decipher is the substrate those systems require to operate cumulatively on human experience.

Neither argument alone is sufficient to justify building this now. Together, they are.

The Bayesian Organisation

The deepest way to understand what Decipher is building toward is through the lens of organisational learning.

A Bayesian system improves by forming a belief, acting, observing the result, and updating the belief. Most organisations approximate the first step and occasionally attempt the second. They almost never observe the result in a structured way and effectively never update a persistent belief — because the belief has no persistent form. Each cycle begins from scratch.

Decipher makes organisational Bayesian updating structurally possible for the first time in the experience domain.

The SEU is the belief. The OU and PDCA cycle are the action. The outcome evaluation is the observation. The IME is the update. The three laws ensure the belief persists, the action is traceable, the observation is causally linked, and the update compounds.

When this loop closes reliably, the organisation stops describing its experience surfaces and starts knowing them. That transition — from episodic observation to cumulative knowledge — is the value the architecture is designed to deliver.

What This Is Not

Decipher is not a CX platform. It does not replace existing analytics, VoC programmes, or survey infrastructure. Those systems continue to generate the signals that feed it.

Decipher is not an AI product. LLMs are used at the ingestion layer and within the PDCA execution cycle. The architecture's value is structural, not inferential.

Decipher is not a dashboard or a reporting tool. It produces no commentary, no narrative summaries, no insight decks.

Decipher is the missing computational layer between human experience signals and organisational action. It is the substrate that makes experience drivers persistent, stateful, actionable, measurable, and cumulative. It is infrastructure. And like all foundational infrastructure, its value becomes invisible once it exists — because without it, nothing above it works properly.

The Category of Innovation

Codd's relational model is an instructive comparison, and the comparison is narrower than the word "like" usually implies.

In 1970, data existed in application-specific formats. Every system had custom query logic. There was no standard structure, no portability, no shared truth.

Codd's contribution was not a better database. It was the formal definition of a primitive, a set of invariants, and a query model that made data computation possible across applications and organisations. The result was not a product. It was infrastructure that the entire software industry reorganised around.

The structural move Decipher is making is analogous in one specific respect: the definition of a primitive, the enforcement of invariants over that primitive, and the provision of a query model that makes a previously informal domain computationally addressable. Experience drivers, which organisations have intuitively centred on for twenty years without being able to operationalise, become first-class entities in a relational substrate.

The claim is not that Decipher will reshape software the way Codd's model did. Human experience is a narrower domain than data in general. The claim is that the same *kind* of move — formalising a primitive and enforcing invariants around it — is what makes the domain finally tractable.

Current State

The architecture is conceptually validated and computationally implemented at prototype stage across:

- Vertical-specific ingestion and mention extraction
- SEU computation engine
- OU clustering and generation logic
- Problem statement generation with full source lineage
- PDCA cycle execution framework with enterprise knowledge base integration architecture
- Closed-loop elasticity computation
- Institutional memory schema
- End-to-end simulation across healthcare and financial services verticals

Production API, enterprise SDK, and multi-tenant deployment are the next phase. The architecture is not a proposal. It is a working model awaiting production engineering and the first enterprise pilot that will generate the empirical proof the foundation deserves.

The Question This Work Raises

If organisations already know that drivers are the real determinants of human experience — why are those drivers still treated as disposable analytical artifacts rather than governed operational entities?

Everything in this architecture is an answer to that question.

The primitive is defined. The invariants are enforced. The query model exists. What remains is the empirical work of proving, at enterprise scale, that a system built on these foundations delivers the compound learning the foundations make possible.

That is the work ahead.

— Raheem Asghar

For technical architecture detail, schema specification, vertical implementation examples, and execution layer documentation — available on request.
